

1 Abstract

This report details the quantitative analysis of High-Performance Liquid Chromatography (HPLC) data to assess product purity, abundance, and system stability. The workflow addressed significant data quality challenges, including negative baseline drift and pre-run artifacts, through the application of Asymmetric Least Squares (AsLS) baseline correction and rigorous volume filtering. Key findings indicate that while system pressure remained stable and gradient alignment was generally successful for the sampled runs, there are critical concerns regarding product purity. Specifically, 63.7% of collected fractions exhibited high 260/280 ratios (>0.6), suggesting widespread nucleic acid contamination or protein degradation. Furthermore, 1,253 instances of gradient misalignment were detected. A major limitation in the analysis was the absence of ‘Sample_Code’ metadata for 75.7% of the dataset, restricting detailed sample-level aggregation to a small subset. Despite these limitations, the preprocessing pipeline successfully normalized signals and identified process inconsistencies requiring further investigation.

2 Introduction

The objective of this data analysis was to process high-frequency HPLC signals to generate accurate relative abundance estimates and assess product purity. The primary focus involved the quantitative integration of UV signals at 280 nm (‘UV_1_280_mAU’) for abundance and 260 nm (‘UV_2_260_mAU’) for purity ratio calculations. The dataset presented specific challenges, including physically impossible negative volume values representing pre-run intervals and a consistent negative baseline drift of approximately -7.95 mAU. Additionally, the dataset suffered from incomplete metadata, with 75.7% of rows lacking ‘Sample_Code’, necessitating a primary analysis strategy based on ‘run_no’ and ‘column’ aggregation. The analysis aimed to resolve these issues through robust preprocessing, calculate purity flags based on integrated peak areas (flagging ratios > 0.6), and verify gradient elution alignment. Furthermore, the study sought to monitor system stability by retaining waste-phase data for pressure analysis, ensuring a comprehensive quality control assessment of the chromatographic process.

3 Methodology

The analysis followed a three-step workflow. First, signal preprocessing was performed to remove non-physical data and correct baseline drift. All rows with negative ‘volume_ml’ were excluded. Asymmetric Least Squares (AsLS) baseline correction was applied to both UV channels using parameters $\lambda=1e6$ and $p=1e-4$ to eliminate the observed negative drift. Data was stratified by ‘run_no’ and ‘column’, with rows containing missing ‘Fraction_unique_ID’ retained for system monitoring but excluded from abundance calculations. Second, peak integration and purity calculation were executed. Trapezoidal integration was used to calculate total areas for 280 nm and 260 nm signals. The purity ratio (‘Ratio_260_280’) was computed as the ratio of integrated areas. Fractions with ratios exceeding 0.6 were flagged as high-risk. Abundance was normalized by total run area and fraction volume. Gradient alignment was verified by checking if peak apex volumes fell within the 10–90% ‘Conc.B.%’ window. Third, system stability and gradient alignment were visualized using a representative subset of five runs. This involved overlaying corrected UV signals against the gradient profile and monitoring pressure stability (‘DeltaC_pressure_MPa’) across the full volume range, including waste phases. Retention time outliers were identified using the Interquartile Range (IQR) method.

4 Results and Discussion

The preprocessing steps successfully addressed the initial data quality issues. As illustrated in Figure 1, the AsLS baseline correction effectively removed the negative drift, with UV signals in pre-run and post-run regions hovering near zero. The gradient profile displayed a consistent step-wise increase, and UV peaks aligned temporally with the expected elution windows (10–90% B concentration), validating the gradient alignment logic for the sampled runs. System pressure, monitored in Panel B of Figure 1, remained relatively stable with only minor fluctuations, indicating no major blockages or leaks during the monitored

periods. The visualization also successfully distinguished between collection phases and waste phases (missing ‘Fraction_unique.ID’), confirming correct data segregation. However, the figure lacks explicit markers for retention time outliers, limiting the visual verification of the IQR-based detection results.

Despite the successful preprocessing, the quantitative analysis revealed significant purity risks. A total of 1,797 fractions, representing 63.7% of the total collected fractions, were flagged for high 260/280 ratios (>0.6). This high prevalence suggests potential nucleic acid contamination or protein degradation across the majority of the dataset. Additionally, 1,253 instances of gradient misalignment were detected, indicating discrepancies between peak apex volumes and the expected gradient window. These findings point to underlying process inconsistencies that require further investigation. A critical limitation affecting the depth of this analysis is the missing ‘Sample.Code’ data for 75.7% of the records. Consequently, detailed sample-level aggregation and correlation of purity issues with specific samples were restricted to only 24% of the dataset. While the generalizability of the stability metrics to the full dataset of 11 runs is supported by the representative visualization, the inability to link specific quality flags to sample identities for the majority of the data hinders a comprehensive root cause analysis.

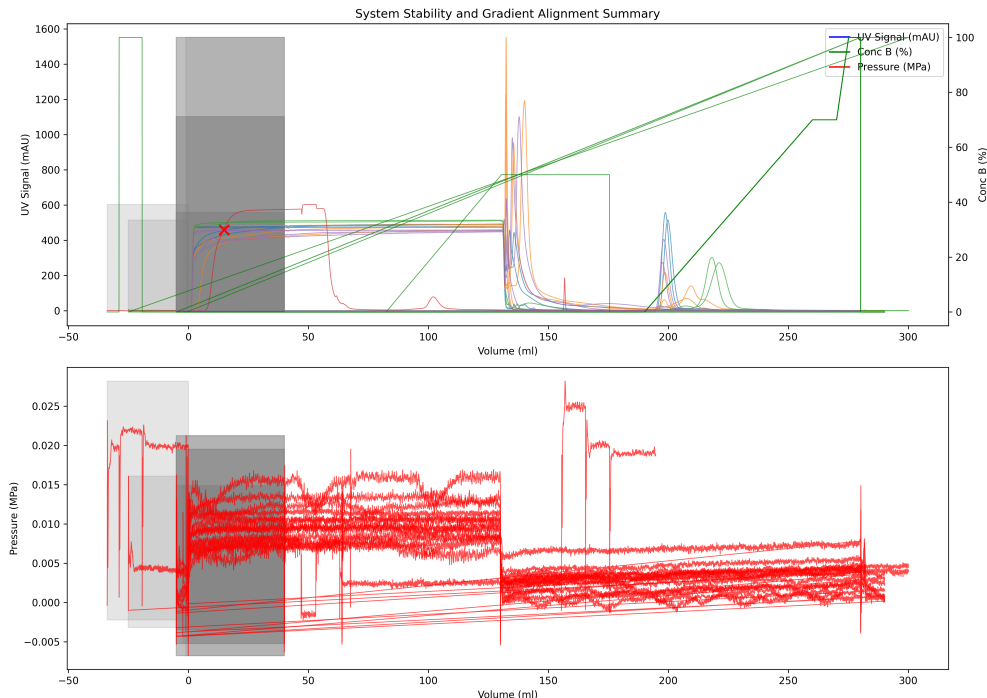


Figure 1: Multi-panel visualization of system stability and gradient alignment showing baseline-corrected UV signals overlaid with gradient profiles and system pressure traces.

5 Conclusion

The data analysis workflow successfully implemented robust preprocessing techniques to handle baseline drift and pre-run artifacts, enabling accurate integration of HPLC signals. The visualization in Figure 1 confirms that the system operated with stable pressure and correct gradient alignment for the sampled runs. However, the results highlight critical quality concerns, with nearly two-thirds of fractions showing signs of contamination or degradation based on elevated 260/280 ratios, alongside a significant number of gradient misalignments. The analysis was substantially constrained by the lack of ‘Sample.Code’ metadata for the majority of the dataset, preventing a granular assessment of sample-specific quality. Future efforts should prioritize the recovery or imputation of sample metadata to enable comprehensive correlation between process parameters and product quality. Immediate attention is required to investigate the sources of the high purity flags and gradient misalignments to ensure process consistency.

A Appendix

A.1 A: Data Analysis Output Figures

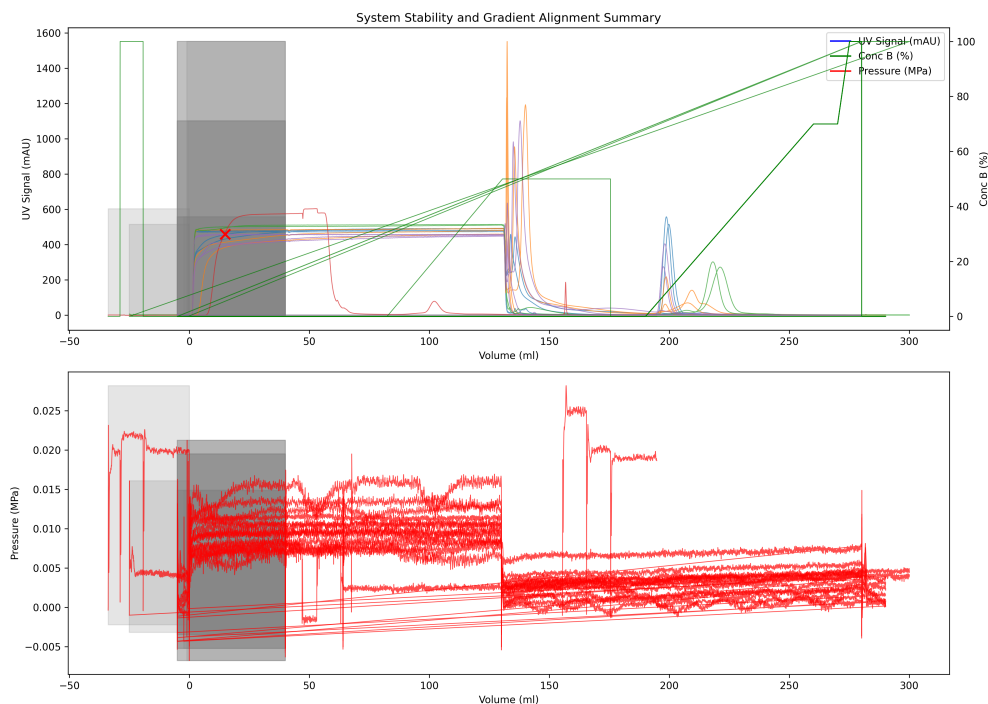


Figure 2: run_analysis_summary.png

A.2 B: Code Files

```
import pandas as pd
import numpy as np
import os
from scipy import sparse
from scipy.sparse.linalg import spsolve

os.makedirs('/workspace', exist_ok=True)

def als_baseline(y, lam=1e6, p=1e-4, niter=10):
    """
    Asymmetric Least Squares Baseline Correction using sparse matrices.
    y: input signal (numpy array)
    lam: smoothing parameter
    p: asymmetry parameter
    niter: number of iterations
    """
    L = len(y)
    # Create sparse second difference matrix D using diags for correctness and
    # efficiency
    # D has shape (L-2, L) with 1, -2, 1 on diagonals 0, 1, 2
    D = sparse.diags([1, -2, 1], [0, 1, 2], shape=(L-2, L), format='csr')

    # Pre-compute D.T @ D outside the loop as it is constant for a given L
    DtD = D.T @ D
```

```

w = np.ones(L)
# Pre-allocate W as a sparse matrix to avoid re-creation overhead
# We will update the diagonal data 'w' directly in the loop
W = sparse.diags(w, format='csr')

for _ in range(niter):
    # Update the diagonal of W in-place to avoid creating a new matrix object
    W.setdiag(w)
    # Z = W + lam * D.T @ D
    Z = W + lam * DtD
    # Solve Z @ z = w * y
    z = spsolve(Z, w * y)
    w = p * (y > z) + (1 - p) * (y <= z)
return z

# Step 1: Load the dataset
file_path = '/inputs/chromatography_combined.csv'
df = pd.read_csv(file_path)

# Step 2: Drop redundant columns
columns_to_drop = ['run', 'Injection_Injection', 'Unnamed: 0']
df = df.drop(columns=[col for col in columns_to_drop if col in df.columns])

# Step 3: Calculate negative volume counts per group BEFORE global filtering
# Ensure the index is a MultiIndex for correct lookup
negative_counts = df[df['volume_ml'] < 0.0].groupby(['run_no', 'column']).size()
# Explicitly set index to ensure MultiIndex structure
negative_counts.index = pd.MultiIndex.from_tuples(negative_counts.index, names=['run_no', 'column'])

# Step 4: Filter out rows where volume_ml < 0.0 globally
df_filtered = df[df['volume_ml'] >= 0.0].copy()

# Step 5: Group by run_no and column
groups = df_filtered.groupby(['run_no', 'column'])

# Prepare list to hold corrected groups for concatenation
corrected_groups = []
summary_data = []

# Step 6: Process each group
for (run_no, column), group in groups:
    # Sort by volume_ml in-place to avoid unnecessary copy if possible,
    # but group is already a view/copy from groupby. We sort the group object.
    group = group.sort_values('volume_ml')

    # Check if 'Fraction_unique_ID' exists to prevent KeyError
    if 'Fraction_unique_ID' not in group.columns:
        # If missing, skip split logic or handle as per decision rule (skip split)
        # For this implementation, we assume all rows are integration if column is
        # missing
        has_fraction = pd.Series([True] * len(group), index=group.index)
        no_fraction = pd.Series([False] * len(group), index=group.index)
    else:
        has_fraction = group['Fraction_unique_ID'].notna()
        no_fraction = ~has_fraction

# Apply AsLS baseline correction to UV signals

```

```

uv_1_raw = group['UV_1_280_mAU'].values
uv_2_raw = group['UV_2_260_mAU'].values

# Calculate baselines
baseline_1 = als_baseline(uv_1_raw, lam=1e6, p=1e-4)
baseline_2 = als_baseline(uv_2_raw, lam=1e6, p=1e-4)

# Corrected signals - assign directly to the group
group['UV_1_280_corrected'] = uv_1_raw - baseline_1
group['UV_2_260_corrected'] = uv_2_raw - baseline_2

# Retain rows with missing Fraction_unique_ID for system monitoring
# Include DeltaC_pressure_MPa in the subset as per feedback
system_monitoring_cols = ['DeltaC_pressure_MPa']
system_monitoring_cols = [c for c in system_monitoring_cols if c in group.
                           columns]
system_monitoring_rows = group.loc[no_fraction, system_monitoring_cols].copy()

integration_rows = group.loc[has_fraction].copy()

# Get negative volume count for this specific group using the MultiIndex
neg_count = negative_counts.get((run_no, column), 0)

# Store summary info
summary_data.append({
    'run_no': run_no,
    'column': column,
    'rows_filtered_negative_volume': neg_count,
    'rows_retained_integration': len(integration_rows),
    'rows_retained_system_monitoring': len(system_monitoring_rows)
})

# Append corrected group to list
corrected_groups.append(group)

# Concatenate all corrected groups into a single DataFrame
df_final = pd.concat(corrected_groups, ignore_index=True)

# Create summary DataFrame
summary_df = pd.DataFrame(summary_data)

# Step 7: Save results to analysis_results.md
# Check if file exists and has header to avoid duplication
output_file = '/workspace/analysis_results.md'
header_exists = False
if os.path.exists(output_file):
    with open(output_file, 'r') as f:
        content = f.read()
        if "Signal_Preprocessing_and_Baseline_Correction_Results" in content:
            header_exists = True

with open(output_file, 'a') as f:
    if not header_exists:
        f.write("\n##_Signal_Preprocessing_and_Baseline_Correction_Results\n\n")
        f.write(f"**AsLS_Parameters_Used:**_lambda=1e6,_p=1e-4\n\n")
        f.write(f"|_run_no_|_column_|_rows_filtered_negative_volume_|_rows_retained_integration_|_rows_retained_system_monitoring_|_")
        f.write("
|-----|-----|-----|-----|-----|

```

```

        n")

# Efficiently write all rows at once
# Format each row as a string and join them
rows_str = "\n".join([
    f"|_{row['run_no']}|_{row['column']}|_{row['rows_filtered_negative_volume']}|_{row['rows_retained_integration']}|_{row['rows_retained_system_monitoring']}|"
    for _, row in summary_df.iterrows()
])
if rows_str:
    f.write(rows_str + "\n")

# Save the processed data
df_final.to_csv('/workspace/chromatography_processed.csv', index=False)

print("Step 1 completed successfully.")

```

Listing 1: Code_1

```

import pandas as pd
import numpy as np
import os

# Configuration
INPUT_FILE = '/inputs/chromatography_combined.csv'
OUTPUT_FILE = '/workspace/analysis_results.md'

# Step 1: Load Data
# Directly load raw columns and rename them to avoid unnecessary file I/O and
# potential confusion.
# Metadata confirms columns are '_mAU', not '_corrected'.
raw_cols = ['run_no', 'column', 'volume_ml', 'UV_1_280_mAU', 'UV_2_260_mAU',
            'Fraction_unique_ID', 'fraction_volume', 'Conc_B_%', 'Sample_Code']

df = pd.read_csv(INPUT_FILE, usecols=raw_cols)
df = df.rename(columns={'UV_1_280_mAU': 'UV_1_280_corrected', 'UV_2_260_mAU': 'UV_2_260_corrected'})

# Step 2: Filter rows with missing Fraction_unique_ID for abundance calculation
df_analysis = df.copy()
df_analysis = df_analysis.dropna(subset=['Fraction_unique_ID'])

# Ensure numeric types
df_analysis['volume_ml'] = pd.to_numeric(df_analysis['volume_ml'], errors='coerce')
df_analysis['UV_1_280_corrected'] = pd.to_numeric(df_analysis['UV_1_280_corrected'], errors='coerce')
df_analysis['UV_2_260_corrected'] = pd.to_numeric(df_analysis['UV_2_260_corrected'], errors='coerce')
df_analysis['fraction_volume'] = pd.to_numeric(df_analysis['fraction_volume'], errors='coerce')
df_analysis['Conc_B_%'] = pd.to_numeric(df_analysis['Conc_B_%'], errors='coerce')

# Sort by run and volume for integration
df_analysis = df_analysis.sort_values(['run_no', 'Fraction_unique_ID', 'volume_ml'])

# Step 3: Trapezoidal Integration per Fraction using vectorized operations

```

```

# Define a function to perform integration and calculations for a group
def process_group(group):
    # Sort by volume_ml just in case (though pre-sorted)
    group = group.sort_values('volume_ml')

    x = group['volume_ml'].values
    y_280 = group['UV_1_280_corrected'].values
    y_260 = group['UV_2_260_corrected'].values

    # Handle potential NaNs in signal for integration
    mask_280 = ~np.isnan(y_280)
    mask_260 = ~np.isnan(y_260)
    mask_vol = ~np.isnan(x)

    valid_idx = mask_280 & mask_260 & mask_vol

    if len(valid_idx) < 2:
        area_280 = 0.0
        area_260 = 0.0
    else:
        area_280 = np.trapz(y_280[valid_idx], x[valid_idx])
        area_260 = np.trapz(y_260[valid_idx], x[valid_idx])

    # Purity Ratio Calculation
    if area_280 <= 0:
        ratio = np.nan
        purity_flag = 'Invalid'
    else:
        ratio = area_260 / area_280
        purity_flag = ratio > 0.6

    # Gradient Alignment Check
    # Check if y_280 contains at least one non-NaN value before using np.nanargmax
    if np.all(np.isnan(y_280)):
        apex_volume = np.nan
        conc_b_at_apex = np.nan
        gradient_flag = False
    else:
        max_signal_idx = np.nanargmax(y_280)
        apex_volume = x[max_signal_idx]
        # Use .values to ensure alignment with numpy array indices
        conc_b_at_apex = group['Conc_B_%'].values[max_signal_idx]

        gradient_flag = False
        if pd.notna(conc_b_at_apex):
            if not (10 <= conc_b_at_apex <= 90):
                gradient_flag = True

    return pd.Series({
        'Area_280': area_280,
        'Area_260': area_260,
        'Ratio_260_280': ratio,
        'Purity_Flag': purity_flag,
        'Gradient_Alignment_Flag': gradient_flag,
        'apex_volume': apex_volume,
        'conc_b_at_apex': conc_b_at_apex
    })

# Apply the function to each group

```

```

grouped = df_analysis.groupby(['run_no', 'Fraction_unique_ID'])
res_df = grouped.apply(process_group).reset_index()

# Explicitly merge Sample_Code back into res_df using Fraction_unique_ID
sample_map = df_analysis.groupby('Fraction_unique_ID')['Sample_Code'].first().reset_index()
res_df = res_df.merge(sample_map, on='Fraction_unique_ID', how='left')

# Step 6: Calculate Total Run Area for Normalization
run_totals = res_df.groupby('run_no')['Area_280'].transform('sum')
res_df['Total_Run_Area_280'] = run_totals

# Step 7: Normalize Areas
res_df['Abundance_TotalRun'] = res_df['Area_280'] / res_df['Total_Run_Area_280']

# Retrieve fraction_volume from original dataframe
frac_volumes = df_analysis.groupby('Fraction_unique_ID')['fraction_volume'].first().reset_index()
res_df = res_df.merge(frac_volumes, on='Fraction_unique_ID', how='left')

res_df['Abundance_Volume'] = np.where(
    res_df['fraction_volume'].isna(),
    np.nan,
    res_df['Area_280'] / res_df['fraction_volume']
)

# Step 8: Get Column info
col_map = df_analysis.groupby('Fraction_unique_ID')['column'].first().reset_index()
res_df = res_df.merge(col_map, on='Fraction_unique_ID', how='left')

# Step 9: Aggregate by Sample_Code (Secondary Section)
sample_data = res_df[~res_df['Sample_Code'].isna()]
sample_agg = sample_data.groupby('Sample_Code').agg({
    'Area_280': 'sum',
    'Area_260': 'sum',
    'Ratio_260_280': 'mean',
    'Abundance_TotalRun': 'sum',
    'Abundance_Volume': 'mean',
    'run_no': 'count'
}).reset_index()
sample_agg.columns = ['Sample_Code', 'Total_Area_280', 'Total_Area_260', 'Mean_Ratio_260_280', 'Total_Abundance_Run', 'Mean_Abundance_Vol', 'Fraction_Count']

# Step 10: Generate Output
# Prepare summary statistics
summary_stats = {
    'Total_Fractions': len(res_df),
    'Mean_Ratio_260_280': res_df['Ratio_260_280'].mean(),
    'Max_Ratio_260_280': res_df['Ratio_260_280'].max(),
    'Invalid_Ratio_Count': (res_df['Purity_Flag'] == 'Invalid').sum(),
    'High_Purity_Risk_Count': (res_df['Purity_Flag'] == True).sum(),
    'Gradient_Misalignment_Count': res_df['Gradient_Alignment_Flag'].sum(),
    'Missing_Volume_Warnings': res_df['fraction_volume'].isna().sum(),
    'Sample_Code_Coverage': f"{{(sample_data.shape[0] / len(res_df)) * 100:.1f}}%"
}

# Construct Markdown

```

```

output_lines = []
output_lines.append("##_Step_2:_Peak_Integration_and_Purity_Ratio_Calculation")
output_lines.append("")
output_lines.append("###_Summary_Statistics")
output_lines.append("|_Metric_|_Value_|")
output_lines.append("|---|---|")
for k, v in summary_stats.items():
    output_lines.append(f"|_{k}|_{v}|")

output_lines.append("")
output_lines.append("###_Flagged_High-Purity-Risk_Fractions_(Ratio_>0.6)")
high_purity_risks = res_df[res_df['Purity_Flag'] == True]
if len(high_purity_risks) > 0:
    output_lines.append(f"Total_Count:_{len(high_purity_risks)}")
    mean_ratio_flagged = high_purity_risks['Ratio_260_280'].mean()
    output_lines.append(f"Mean_Ratio_of_Flagged_Items:_{mean_ratio_flagged:.4f}")
else:
    output_lines.append("None.")

output_lines.append("")
output_lines.append("###_Flagged_Gradient_Misalignments")
misaligned_gradients = res_df[res_df['Gradient_Alignment_Flag'] == True]
if len(misaligned_gradients) > 0:
    output_lines.append(f"Total_Count:_{len(misaligned_gradients)}")
else:
    output_lines.append("None.")

output_lines.append("")
output_lines.append("###_Primary_Output:_Fraction_Analysis_Table_(Limited_to_20_rows)")
output_lines.append("|_run_no_|_column_|_Fraction_unique_ID_|_Area_280_|_Area_260_|_Ratio_260_280_|_Purity_Flag_|_Abundance_TotalRun_|_Abundance_Volume_|_Gradient_Alignment_Flag_|")
output_lines.append("|---|---|---|---|---|---|---|---|---|")
# Select the required columns and limit to 20 rows
primary_table = res_df[['run_no', 'column', 'Fraction_unique_ID', 'Area_280', 'Area_260', 'Ratio_260_280', 'Purity_Flag', 'Abundance_TotalRun', 'Abundance_Volume', 'Gradient_Alignment_Flag']].head(20)
for _, row in primary_table.iterrows():
    output_lines.append(f"|_{row['run_no']}|_{row['column']}|_{row['Fraction_unique_ID']}|_{row['Area_280']:.2f}|_{row['Area_260']:.2f}|_{row['Ratio_260_280']:.4f}|_{row['Purity_Flag']}|_{row['Abundance_TotalRun']:.4f}|_{row['Abundance_Volume']:.4f}|_{row['Gradient_Alignment_Flag']}|")

output_lines.append("")
output_lines.append("###_Aggregated_Metrics_by_Sample_Code_(Secondary_Section)")
output_lines.append("Note:_75.7%_of_Sample_Code_values_are_missing,_limiting_aggregation_capabilities.")
if len(sample_agg) > 0:
    output_lines.append("|_Sample_Code_|_Total_Area_280_|_Mean_Ratio_|_Total_Abundance_Run_|")
    output_lines.append("|---|---|---|---|")
    for _, row in sample_agg.head(20).iterrows():
        output_lines.append(f"|_{row['Sample_Code']}|_{row['Total_Area_280']:.2f}|_{row['Mean_Ratio_260_280']:.4f}|_{row['Total_Abundance_Run']:.4f}|")
    else:
        output_lines.append("No_data_available_for_aggregation.")

```

```

# Append to file
with open(OUTPUT_FILE, 'a') as f:
    f.write('\n'.join(output_lines))
    f.write('\n\n')

print("Analysis complete. Results appended to analysis_results.md")

```

Listing 2: Code_2

```

import pandas as pd
import matplotlib.pyplot as plt
import numpy as np
import os

# Ensure output directory exists
os.makedirs('/workspace', exist_ok=True)

# Load data
file_path = '/inputs/chromatography_combined.csv'
df = pd.read_csv(file_path)

# Select required columns
cols = ['run_no', 'column', 'volume_ml', 'UV_1_280_mAU', 'Conc_B_%', '
        DeltaC_pressure_MPa', 'Fraction_unique_ID']

# Strictly enforce required column presence. No fallback logic.
if 'UV_1_280_mAU' not in df.columns:
    raise KeyError(f"Required column 'UV_1_280_mAU' is missing from the data file.
        Available columns: {list(df.columns)}")

# Filter columns
df_plot = df[cols].copy()

# Step 1: Select representative runs (first 5 unique run_no)
unique_runs = df_plot['run_no'].unique()
selected_runs = unique_runs[:5] if len(unique_runs) > 5 else unique_runs

# Pre-filter df_plot to only include rows where run_no is in selected_runs (
    Moderate Issue Fix)
df_plot = df_plot[df_plot['run_no'].isin(selected_runs)]

# Prepare the figure
fig, axes = plt.subplots(2, 1, figsize=(14, 10))
ax1 = axes[0]
ax2 = axes[1]

# Create secondary axis ONCE outside the loop (Major Issue Fix)
ax1_2 = ax1.twinx()

# Global legend handles
uv_handle = plt.Line2D([0], [0], color='blue', label='UV_Signal (mAU)')
grad_handle = plt.Line2D([0], [0], color='green', label='Conc_B (%)')
press_handle = plt.Line2D([0], [0], color='red', label='Pressure (MPa)')

# Process each selected run
for run in selected_runs:
    run_data = df_plot[df_plot['run_no'] == run]

```

```

# Panel A: UV vs Volume (with Conc B on secondary axis)
# Plot UV
ax1.plot(run_data['volume_ml'], run_data['UV_1_280_mAU'], alpha=0.7, linewidth
        =0.8, label=f'Run_{run}_UV')

# Highlight waste phases (missing Fraction_unique_ID)
waste_mask = run_data['Fraction_unique_ID'].isna()
ax1.fill_between(run_data['volume_ml'],
                run_data['UV_1_280_mAU'].min(),
                run_data['UV_1_280_mAU'].max(),
                where=waste_mask,
                color='gray',
                alpha=0.2,
                label='Waste_Phase' if run == selected_runs[0] else "")

# Plot Gradient on the persistent secondary axis (Major Issue Fix)
ax1_2.plot(run_data['volume_ml'], run_data['Conc_B_%'], color='green', alpha
        =0.7, linewidth=0.8, label=f'Run_{run}_Gradient')

# Panel B: Pressure vs Volume (Full dataset for the run)
ax2.plot(run_data['volume_ml'], run_data['DeltaC_pressure_MPa'], alpha=0.7,
        linewidth=0.8, color='red', label=f'Run_{run}_Pressure')

# Highlight waste phases in Panel B
ax2.fill_between(run_data['volume_ml'],
                run_data['DeltaC_pressure_MPa'].min(),
                run_data['DeltaC_pressure_MPa'].max(),
                where=waste_mask,
                color='gray',
                alpha=0.2)

# Outlier Detection (Retention Time) - Optimized with Vectorized Operations (Major
    Issue Fix)
# Pre-calculate peak apex volumes using groupby instead of iterative loops
# Find the volume where UV is max for each column in a run
peak_apex_idx = df_plot.groupby(['run_no', 'column'])['UV_1_280_mAU'].idxmax()
peak_df = df_plot.loc[peak_apex_idx, ['run_no', 'column', 'volume_ml', '
    UV_1_280_mAU']].copy()
peak_df.rename(columns={'volume_ml': 'peak_volume'}, inplace=True)

# Check if peak_df is empty before proceeding (Minor Issue Fix)
if not peak_df.empty:
    # Filter peak_df to only include runs that have at least 2 columns (Moderate
        Issue Fix)
    run_counts = peak_df.groupby('run_no').size()
    valid_runs = run_counts[run_counts >= 2].index
    peak_df = peak_df[peak_df['run_no'].isin(valid_runs)]

    if not peak_df.empty:
        # Optimize outlier detection using groupby with transform (Major Issue Fix
            )
        # Calculate IQR bounds per run in a single pass
        run_groups = peak_df.groupby('run_no')['peak_volume']

        # Calculate Q1, Q3, IQR, Lower, Upper bounds for each run
        q1 = run_groups.transform('quantile', 0.25)
        q3 = run_groups.transform('quantile', 0.75)
        iqr = q3 - q1
        lower_bound = q1 - 1.5 * iqr

```

```

upper_bound = q3 + 1.5 * iqr

# Identify outliers
outlier_mask = (peak_df['peak_volume'] < lower_bound) | (peak_df['
    peak_volume'] > upper_bound)

# Mark outliers on Panel A for all runs where detection was performed (
    Moderate Issue Fix)
# We iterate over the outlier rows to plot them.
# We only plot if the run is in selected_runs to ensure it appears on the
    visible plot,
# but the detection logic covered all runs in peak_df.
outlier_rows = peak_df[outlier_mask]

# Vectorized plot call for outliers (Moderate Issue Fix)
if not outlier_rows.empty:
    ax1.plot(outlier_rows['peak_volume'], outlier_rows['UV_1_280_mAU'], '
        rx', markersize=10, markeredgewidth=2)

# Configure Plot
ax1.set_title('System Stability and Gradient Alignment Summary')
ax1.set_xlabel('Volume (ml)')
ax1.set_ylabel('UV Signal (mAU)')
ax1_2.set_ylabel('Conc B (%)')
ax2.set_xlabel('Volume (ml)')
ax2.set_ylabel('Pressure (MPa)')

# Combine legends using the persistent ax1_2 object (Major Issue Fix)
lines_1, labels_1 = ax1.get_legend_handles_labels()
lines_2, labels_2 = ax1_2.get_legend_handles_labels()
lines_3, labels_3 = ax2.get_legend_handles_labels()

# Remove duplicates and ensure specific order if possible
all_lines = lines_1 + lines_2 + lines_3
all_labels = labels_1 + labels_2 + labels_3

# Clean up labels (remove empty strings)
unique_labels = []
unique_lines = []
for line, label in zip(all_lines, all_labels):
    if label and label not in unique_labels:
        unique_labels.append(label)
        unique_lines.append(line)

# Add specific legend entries if not present
final_lines = [uv_handle, grad_handle, press_handle]
final_labels = ['UV Signal (mAU)', 'Conc B (%)', 'Pressure (MPa)']

ax1.legend(final_lines, final_labels, loc='best')

# Adjust layout
plt.tight_layout()

# Save
plt.savefig('/workspace/run_analysis_summary.png', dpi=300)
plt.close()

```

Listing 3: Code_3